5.TCP And UDP

What TCP Is and Why It Exists

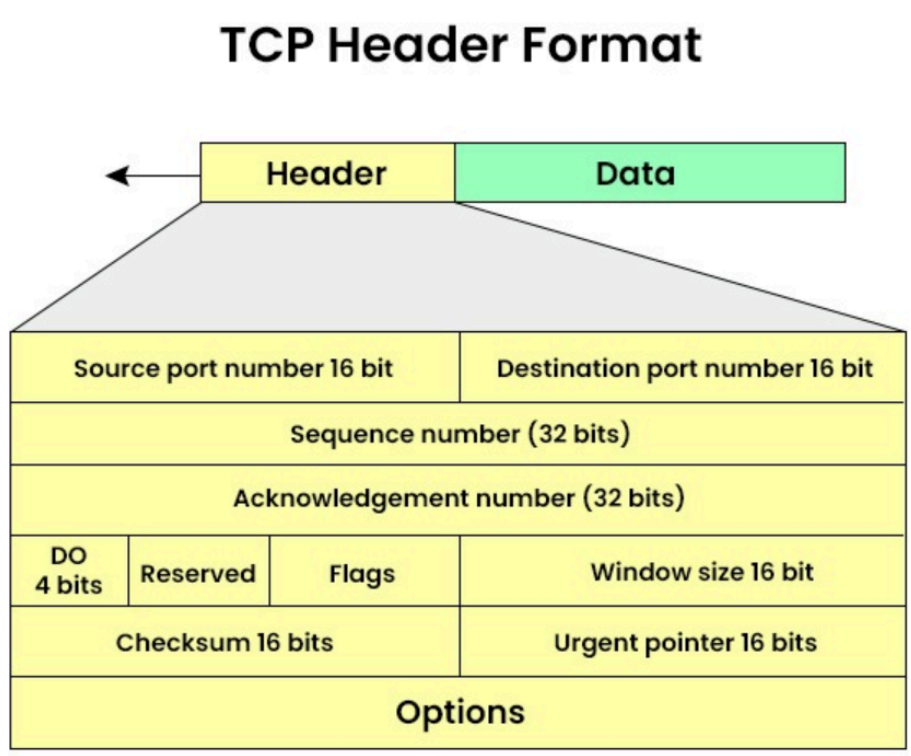
- IP delivers packets to a machine — but it is unreliable, unordered, and connectionless
- TCP sits on top of IP and solves three problems IP does not: reliability, ordering, and flow control
- TCP is a protocol at the transport layer — it does not care what the data is, it just ensures it arrives correctly
- TCP treats data as a continuous stream of bytes, not individual packets — this is important to understand internally

Core Properties of TCP

- Connection-oriented — a connection must be established before any data is sent
- Reliable — every byte sent is acknowledged, unacknowledged bytes are retransmitted
- Ordered — data is delivered to the application in the exact order it was sent
- Flow controlled — sender cannot overwhelm the receiver
- Congestion controlled — sender adjusts speed based on network conditions
- Full-duplex — both sides can send and receive simultaneously on the same connection

What a TCP Connection Actually Is

- A TCP connection is not a physical wire or a dedicated channel
- It is a shared state maintained by both the client and server in memory
- Both sides store: sequence numbers, acknowledgement numbers, window sizes, connection state
- The OS manages this state — it is stored in a structure called a TCB (Transmission Control Block)
- If either machine crashes, the other side still thinks the connection is open — this is called a half-open connection

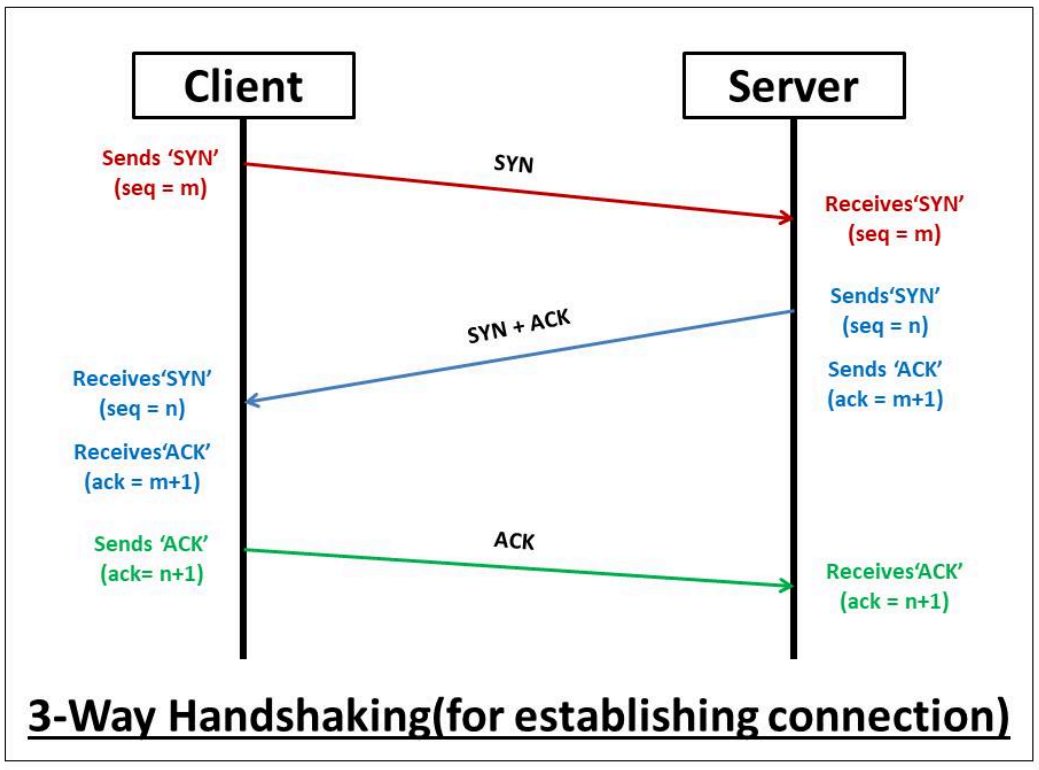


Sequence & Acknowledgement Numbers

- Every byte in a TCP stream has a sequence number — 32-bit integer, wraps around after ~4 billion
- ISN (Initial Sequence Number) is randomly chosen at connection start — prevents old packets from being mistaken for new data and prevents sequence prediction attacks
- ACK number means "I have received everything up to this byte, send me the next one" — ACK = last received seq + 1
- ACKs are cumulative — one ACK can cover multiple segments
- SACK (Selective ACK) — an extension that lets the receiver say exactly which bytes arrived and which are missing, so the sender retransmits only what is lost

The 3-Way Handshake — In Depth

This is the process of establishing a TCP connection. Three messages are exchanged before any application data flows.



Step 1 — SYN (Client to Server)

- Client picks a random ISN — let's say 1000
- Client sends a TCP segment with the SYN flag set
- SYN flag means "I want to synchronize sequence numbers with you"
- The segment contains: SYN=1, Seq=1000
- No data is sent in this segment, just the control information
- Client moves to state: SYN_SENT

Step 2 — SYN-ACK (Server to Client)

- Server receives the SYN, it now knows the client's ISN is 1000
- Server picks its own random ISN — let's say 5000
- Server sends back a segment with both SYN and ACK flags set
- The segment contains: SYN=1, ACK=1, Seq=5000, Ack=1001
- Ack=1001 means "I received up to byte 1000, send me byte 1001 next"
- Server moves to state: SYN_RECEIVED
- Why does the server also send a SYN — because TCP is full-duplex, both sides need to establish their own sequence numbers independently

Step 3 — ACK (Client to Server)

- Client receives the SYN-ACK, it now knows the server's ISN is 5000
- Client sends an ACK segment: ACK=1, Seq=1001, Ack=5001
- Ack=5001 means "I received up to byte 5000, send me byte 5001 next"
- Client moves to state: ESTABLISHED
- Server receives this ACK and moves to state: ESTABLISHED
- Connection is now open — application data can flow

What the Handshake Achieves

- Both sides have agreed on starting sequence numbers
- Both sides know the other is reachable and ready
- Both sides have allocated memory and resources for this connection (TCB (Transmission control black) created)
- One full round trip of latency is added before any data can flow — this is the cost

Why 3 Steps and Not 2

- Two steps would mean only the client confirms the server's sequence number
- The server would never confirm it received the client's ACK
- With 3 steps, both sides confirm each other's sequence numbers
- You cannot do it in fewer than 3 steps and still have both sides synchronized

What Happens After the Handshake

- Application data starts flowing — each segment has a sequence number
- Receiver sends ACKs back

- Sender maintains a send buffer and a retransmission timer for unacknowledged data
- Receiver maintains a receive buffer and reorders out-of-order segments before passing to application

TCP Connection States (important for debugging and interviews)

- LISTEN — server is waiting for incoming connections
- SYN_SENT — client sent SYN, waiting for SYN-ACK
- SYN_RECEIVED — server received SYN, sent SYN-ACK, waiting for ACK
- ESTABLISHED — handshake complete, data flowing
- FIN_WAIT_1, FIN_WAIT_2, CLOSE_WAIT, LAST_ACK, TIME_WAIT — teardown states
- TIME_WAIT is important — after connection closes, the port stays in TIME_WAIT for 2 * MSL (Maximum Segment Lifetime, usually 2 minutes) to ensure all delayed packets have disappeared from the network

Common Misconceptions

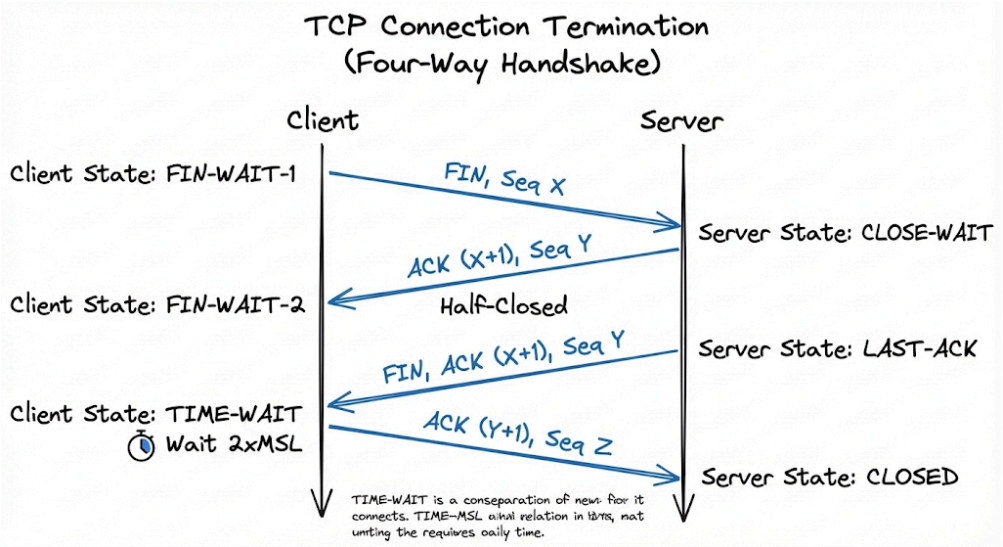
- "TCP guarantees packets arrive" — TCP guarantees the data stream arrives correctly, not individual packets. Packets are dropped and retransmitted transparently. The application never sees this.
- "SYN flood attack" — attacker sends thousands of SYN packets but never sends the final ACK. Server allocates a TCB for each half-open connection, exhausting memory. Mitigation is SYN cookies — server does not allocate state until the final ACK arrives.
- "The handshake only happens once" — correct, but TLS adds its own handshake on top of the TCP handshake. So for HTTPS you pay the TCP handshake cost plus the TLS handshake cost before data flows.

TCP 4-Way Termination

Why 4 Steps and Not 3

- TCP is full-duplex — both sides have an independent stream of data
- Each side must independently close its own stream
- Closing one direction does not close the other
- This is called half-close — one side can stop sending but still receive
- That is why termination takes 4 steps, not 3

The 4-Way Termination In Detail



Assume client initiates the close. Both sides are in ESTABLISHED state.

Step 1 — FIN (Client to Server)

- Client has no more data to send
- Client sends segment with FIN flag set
- FIN means "I am done sending data"
- Client moves to state: FIN_WAIT_1
- Client can still receive data from server — its receive side is still open

Step 2 — ACK (Server to Client)

- Server receives the FIN
- Server sends back ACK immediately
- This tells the client "I got your FIN, acknowledged"
- Server moves to state: CLOSE_WAIT
- Client moves to state: FIN_WAIT_2
- At this point, server can still send data to client — the server's send stream is still open

- Client must stay alive and receive this data

Step 3 — FIN (Server to Client)

- Server finishes sending whatever remaining data it had
- Server sends its own FIN segment
- Server moves to state: LAST_ACK

Step 4 — ACK (Client to Server)

- Client receives server's FIN
- Client sends final ACK
- Client moves to state: TIME_WAIT
- Server receives ACK, moves to state: CLOSED — server is fully done

TIME_WAIT State — Why It Exists and Why It Matters

- Client waits 2 * MSL(Maximum Segment lifetime) (up to 2 minutes) before fully closing
- Reason 1: the final ACK may be lost; server will retransmit FIN and client must respond
- Reason 2: delayed packets from the old connection must expire before the port is reused
- At scale this is a real problem — thousands of ports stuck in TIME_WAIT exhausts the available port range

Reliability — Retransmission

- Sender starts a timer per segment; if ACK is not received before timeout, segment is retransmitted
- `RTO (Retransmission Timeout)` is dynamic — calculated from measured RTT; doubles on each failure (exponential backoff)
- Sender holds all unacknowledged segments in a retransmission buffer until ACK arrives

Fast Retransmit

- Waiting for RTO can take seconds — too slow
- If a segment is lost but later segments arrive, the receiver sends duplicate ACKs for the last good segment
- 3 duplicate ACKs = the sender immediately retransmits the missing segment without waiting for RTO
- Recovers in one RTT instead of waiting for the full timeout

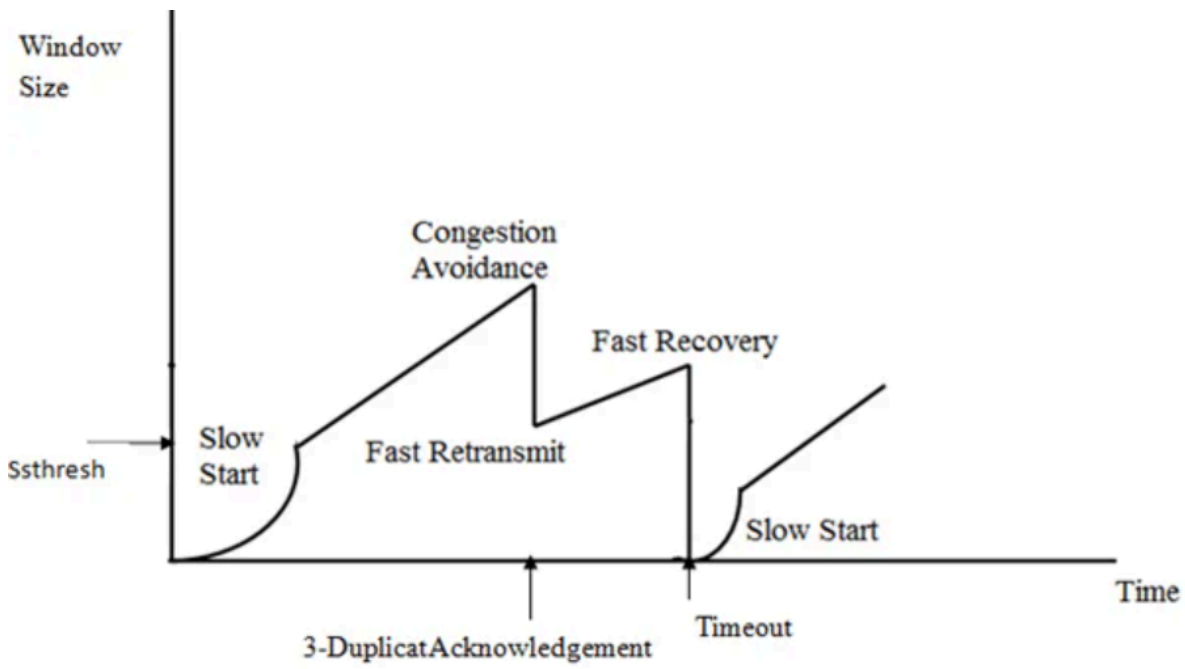
Flow Control

- Waiting for RTO can take seconds — too slow
- If a segment is lost but later segments arrive, the receiver sends duplicate ACKs for the last good segment
- 3 duplicate ACKs = the sender immediately retransmits the missing segment without waiting for RTO
- Recovers in one RTT instead of waiting for the full timeout

Sliding Window

- Without it: send one segment, wait for ACK, send next — extremely inefficient on high-latency links
- With it: sender keeps a window of bytes it can send without waiting for ACKs
- As ACKs come in, the window slides forward — more bytes become available to send
- Keeps the network pipe full at all times

Congestion Control



The Problem Congestion Control Solves

- Flow control protects the receiver
- Congestion control protects the network
- Routers have finite buffer space — if too much data is sent into the network, router buffers fill up and packets are dropped
- TCP must detect this and slow down — otherwise every sender floods the network and everyone suffers

Congestion Window (cwnd)

- TCP maintains a congestion window on the sender side
- cwnd limits how much data the sender can have in flight, independently of the receiver window
- Actual limit on in-flight data = min(rwnd, cwnd)
- rwnd is set by the receiver, cwnd is managed by the sender's congestion control algorithm
- cwnd starts small and grows based on network feedback

Slow Start

- When a connection first opens, the sender does not know the network's capacity
- Slow start begins with cwnd = 1 MSS (Maximum Segment Size — typically 1460 bytes for Ethernet)
- For every ACK received, cwnd increases by 1 MSS
- This means cwnd doubles every round trip — exponential growth
- Round 1: cwnd=1, send 1 segment, get 1 ACK, cwnd=2
- Round 2: cwnd=2, send 2 segments, get 2 ACKs, cwnd=4
- Round 3: cwnd=4, send 4 segments, get 4 ACKs, cwnd=8
- This continues until cwnd reaches a threshold called ssthresh (slow start threshold)

Congestion Avoidance

- After ssthresh: cwnd grows by 1 MSS per RTT — linear, cautious growth
- Continues until loss is detected

On Loss

- 3 duplicate ACKs (mild): ssthresh = cwnd/2, cwnd = ssthresh, skip slow start, go straight to congestion avoidance (Fast Recovery)
- RTO fires (severe): ssthresh = cwnd/2, cwnd reset to 1, full slow start again

Why This Matters in System Design

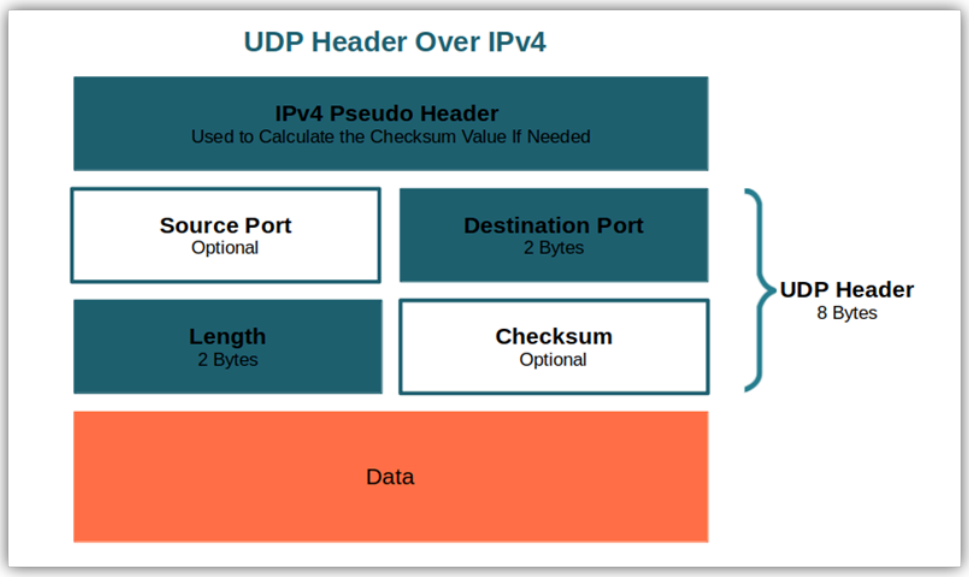
- Every new TCP connection starts slow — cold start problem
- On a connection with 100ms RTT, it takes several round trips just to ramp up to full speed
- This is why HTTP keep-alive and HTTP/2 connection reuse matter — reusing an existing connection avoids slow start
- This is why CDNs are placed geographically close to users — lower RTT means faster ramp-up and higher throughput
- This is why QUIC (HTTP/3) is faster for short connections — it was designed to avoid these slow start penalties
- A single large file transfer benefits from TCP ramping up because the connection lasts long enough
- Many small short-lived connections never escape slow start — each one starts from cwnd=1 again

UDP — User Datagram Protocol

What UDP Is

- UDP is a transport layer protocol just like TCP
- It takes data from the application layer, wraps it in a datagram, and sends it via IP
- That is basically all it does
- No connection setup, no acknowledgements, no ordering, no retransmission, no flow control, no congestion control
- The application sends data and UDP fires it into the network and forgets about it

UDP Header — Why It Is Fast



TCP header is minimum 20 bytes and contains sequence numbers, ACK numbers, window size, flags, checksum, urgent pointer, and options.

UDP header is exactly 8 bytes and contains only 4 fields:

- Source port — 2 bytes
- Destination port — 2 bytes
- Length — 2 bytes
- Checksum — 2 bytes (optional in IPv4, mandatory in IPv6)

That is the entire protocol. No state, no tracking, no memory of previous packets.

How UDP Works

- Application hands data to UDP
- UDP wraps it in a datagram with the 8-byte header
- Datagram is handed to the IP layer which sends it
- If it arrives, it arrives. If it does not, nothing happens.
- If it arrives out of order, UDP passes it to the application out of order
- If it arrives duplicated, UDP passes both copies to the application
- The application is responsible for handling all of these cases if it cares about them

Why UDP Is Faster Than TCP — Specific Reasons

- No handshake — data flows immediately, zero setup latency
- No ACKs — no waiting, no retransmission buffer
- No retransmission — lost packet is gone, sender keeps moving
- No ordering overhead — no buffering for gaps, deliver immediately
- No congestion control — no slow start, sends at full rate from the first datagram
- No connection state — no TCB, no per-connection memory in the OS

No Guarantee of Delivery — What This Actually Means

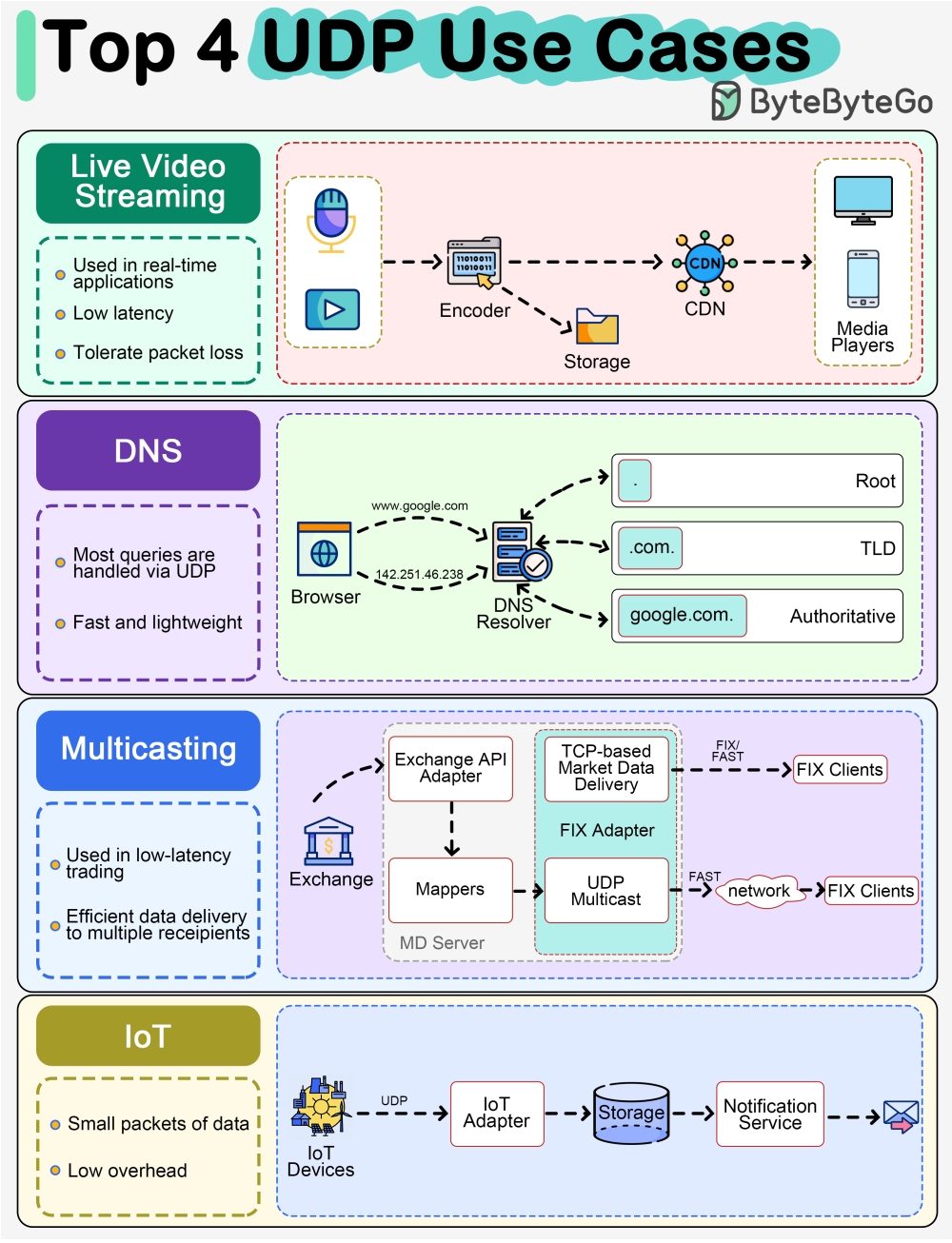
- A UDP datagram can be dropped at any router along the path if the router buffer is full
- A UDP datagram can be dropped by the receiver's OS if its buffer is full
- The sender has no idea the packet was dropped
- The receiver has no idea the sender sent something that did not arrive
- There is no mechanism in UDP itself to detect or recover from this

- If the application needs to know, it must implement its own detection — which some protocols do (QUIC, DTLS, application-level ACKs in games)

No Ordering — What This Actually Means

- Packets can take different routes through the internet and arrive in a different order than they were sent
- TCP hides this from the application completely
- UDP exposes it — the application sees packets in whatever order they arrive
- If you send datagrams 1, 2, 3 — the application might receive 1, 3, 2
- Again, if the application cares, it must handle ordering itself — usually by putting a sequence number in the application-level message

UDP Use Cases — Why Each One Makes Sense



UDP Use Cases

- **DNS** — tiny request/response, fits in one datagram; if response does not arrive, resolver just resends the query — application-level retry is trivial; TCP handshake would double the latency of every single lookup; falls back to TCP only for large responses like zone transfers
- **Live video / audio** — stream is a continuous flow of frames; if a frame is lost, retransmitting it takes at least one RTT by which time playback has already moved past that point; a missing frame causes a brief visual glitch which is acceptable; freezing the entire stream to wait for one frame is not acceptable; UDP lets the stream keep moving regardless of loss
- **Online gaming** — game server sends player position updates dozens of times per second; if update 47 is lost, update 48 and 49 with newer positions are already on the way; retransmitting stale position data is pointless; UDP lets the game keep pushing fresh state without being blocked; games layer thin application-level reliability only for critical events like player death or weapon fired where missing data actually matters
- **QUIC / HTTP3** — QUIC is a full transport protocol built on UDP; it reimplements reliability and ordering but does it per-stream rather than per-connection; a lost packet only blocks the stream it belongs to, not every other stream like TCP does; this eliminates head-of-line blocking entirely; QUIC also handles connection setup and TLS in one round trip instead of TCP and TLS doing separate handshakes

TCP vs UDP — When to Choose What

- Choose TCP when: data must arrive completely and correctly — file transfers, emails, database queries, web pages, API calls
- Choose UDP when: speed and low latency matter more than guaranteed delivery — live video, voice calls, gaming, DNS, IoT sensor data
- Choose UDP and build on top when: you need some reliability but with more control than TCP gives you — QUIC, game engines, financial market data feeds

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection Type	Connection-oriented (requires handshake)	Connectionless (no handshake)
Reliability	Reliable (guarantees delivery, retransmits lost packets)	Unreliable (no guarantee of delivery)
Speed	Slower (due to error checking & acknowledgments)	Faster (no overhead of checks/retransmission)
Data Order	Ensures ordered delivery	No guarantee of order
Use Cases	Web (HTTP/HTTPS), Email, File transfer	Streaming, Gaming, DNS, VoIP